

◆ VishwaTech-Labs ◆

PYTHON SUPER-DUPER MASTER TEST • 50 QUESTIONS

Complete coverage: Data Types & Methods • Operators • Control Flow • Functions • Error & Exception Handling • Iterators & Generators • Modules • Packages • Libraries & Frameworks

Student Name	<u>PAVAN KUMAR</u>	Marks	<u> </u> / 100
Date	<u>16-09-2026</u>	Time	120 min
Batch / Section	<u> </u>	Instructor	<u>Vishwa</u>

INSTRUCTIONS

Answer all 50 questions. MCQs: choose one best answer. Output questions: show exact output. Coding questions: write complete Python 3.x code in the provided space. For debugging, identify the error and correct it. Clean indentation and readable code are expected.

PART A — MULTIPLE CHOICE

01. Which expression returns a float?

- A) 10//3 B) 10/3 C) int(10/3) D) 10%3

02. Which type is immutable?

- A) list B) set C) tuple D) dict

03. What is `(3+4j).conjugate()`?

- A) 3-4j B) 4-3j C) -3+4j D) 3+4j

04. What does `bool(0)` return?

- A) True B) False C) 0 D) Error

05. Which string method removes whitespace from both ends?

- A) strip() B) split() C) trim() D) clean()

06. Which string method combines an iterable of strings?

- A) split() B) join() C) merge() D) concat()

07. Which list method adds one item at the end?

- A) extend() B) append() C) insert() D) add()

08. Which list method adds multiple items from an iterable?

- A) append() B) extend() C) insert() D) update()

09. Which set method removes an item without raising `KeyError` if missing?

- A) remove() B) discard() C) pop() D) delete()

10. Which dictionary method returns key-value pairs?

- A) keys() B) values() C) items() D) pairs()

11. Which tuple method returns the number of occurrences?

- A) count() B) index() C) occurrences() D) find()

12. Which function converts a value to an integer?

- A) str() B) int() C) integer() D) number()

13. Which operator performs exponentiation?

- A) ^ B) ** C) // D) ^^

14. What is the result of `17 % 5`?

- A) 2 B) 3 C) 4 D) 5

15. Which operator checks object identity?

- A) == B) is C) in D) =

16. Which logical operator is true only when both operands are true?

- A) or B) and C) not D) xor

17. Which keyword skips the current loop iteration?

- A) break B) continue C) pass D) skip

18. Which keyword exits a loop immediately?

- A) stop B) continue C) break D) exitloop

19. Which statement creates a conditional expression?

- A) x if condition else y B) if x then y C) choose(x,y) D) condition ? x : y

20. Which parameter syntax collects extra positional arguments?

- A) *args B) *args C) &args; D) args[]

PART B — OUTPUT & DEBUGGING

21. Predict the output:

```
x=[1,2,3]; x.append(4); x.pop(1); print(x)
```

POP will remove item from the given index
output: [1, 3, 4]

22. Predict the output:

```
print('Python'.upper().replace('PY', 'MY'))
```

o/p: MYTHON

23. Predict the output:

```
a={1,2,3}; b={3,4}; print(a.intersection(b))
```

o/p: {3}

24. Predict the output:

```
d={'a':10}; print(d.get('b',99))
```

get(key, default) → Syntax for get Function, it will default value 99 if the key is not present.
o/p: 99

25. Identify the exception raised by `'10' / 0`.

Zero Division Exception

26. Identify the exception raised by `int('abc')`.

Value Error

27. Fix the syntax/logic issue: `if x = 10:`

x = 10 is assignment (=), so 10 is assigned to x for condition you should use x == 10

28. What happens here?

```
it=iter([10,20]); print(next(it)); print(next(it)); print(next(it))
```

Stop Iteration Error

PART C — ERROR & EXCEPTION HANDLING

29. Differentiate Syntax Error, Runtime Error and Logical Error. Give one Python example of each.

① Syntax Error: occurs when syntax is incorrect.
Example: `if 10 > 5`
`Print("10 is greater")` → Here `;` is missing

② Runtime Error: Error at the time of Execution. Ex: `print(10/0)`.

③ Logical Error: program runs fine, But not get the expected output.

30. Explain Error vs Exception. Why is exception handling important in automation/programs?

Error: is like Syntax Error, if program syntax is incorrect, error occur and cannot run the program.

Exception: It stops the program suddenly, to avoid we need to handle that using try, catch.

Example: If you want to open a file and if the file is not found, what you need to inform to user.

31. Write one example demonstrating try + except + else + finally. Explain when each block executes.

try: `a=10, b=2`

`result = a/b`

except ZeroDivisionError:

`print("You cannot divide the number with 0")`

else:

`print(result)`

finally: `print("Division completed")`

32. Write code handling multiple exceptions, for example `ValueError` and `ZeroDivisionError`, with separate except blocks.

try:

`a = int(input("Enter first number:"))`

`b = int(input("Enter second number:"))`

`result = a/b`

`print(result)`

except ValueError:

`print("Please Enter numbers only")`

except ZeroDivisionError: `print("Cannot divide by 0")`

33. Use `raise` to reject an invalid age. Then create and handle a custom `AgeError`.

`class CustomAgeError(Exception):`

`pass`

try:

`age = int(input("Enter your age:"))`

`if age < 18:`

`raise CustomAgeError("Invalid age")`

`print("age is valid")`

34. Explain the Python Exception Hierarchy. Include BaseException, Exception and at least four common subclasses.

PART D — ITERATORS & GENERATORS

35. Explain iterable vs iterator. Demonstrate iter() and next() with a list.

iterable - It is like sequence of values, using iterator, you can get next position and item.

Example: listN = [10, 20, 30]

it = iter(listN)

print(next(it))

print(next(it)) o/p: 10, 20

36. Create a custom iterator class Countdown implementing __iter__() and __next__().

37. Write a generator function even_numbers(n) using yield.

```
def even_numbers(n):  
    for i in range(n+1):  
        if i%2 == 0:  
            yield i
```

```
for number in even_numbers(10):  
    print(number)
```

38. Write a generator expression that produces squares from 1 to 20.

```
def squares():  
    for i in range(1, 21):  
        yield i**2
```

```
for x in squares():  
    print(x)
```

39. Explain StopIteration. When does it occur and how does a for loop handle it?

```
numbers = iter([10, 20, 30])
```

```
print(next(numbers))
```

```
print(next(numbers))
```

```
print(next(numbers))
```

```
print(next(numbers))
```

→ Stop Iteration Err. here

40. Compare Generator vs List in terms of memory, execution and reuse. Give an example.

Generator: Produce one value at the time of "yield". It won't store all value in memory.

Memory: Stores all values in memory. Example: List.

Generator

- ① Generates one value at a time
- ② In terms of execution, all values created ~~immediately~~ when it is required. ✓
- ③ It cannot be reused
- ④ Syntax: yield

List

Stores all value in memory.

All values are created immediately.

It can be reused

Syntax: `l = [ ]`

PART E — MODULES, PACKAGES & LIBRARIES

41. Create a module calculator.py with add(), subtract(), multiply() and show import usage.

① Create a file as calculator.py and save it at the end.

```
def add(a,b):  
    return a+b
```

```
def sub(a,b):  
    return a-b
```

```
def mul(a,b):  
    return a*b
```

② Create another file main.py

```
import calculator
```

```
print(calculator.add(5,5))
```

```
print(calculator.sub(10,5))
```

```
print(calculator.mul(10,5))
```

42. Demonstrate import, from ... import and an alias using 'as'.

③ from ^{calculator} import add
print(add(10,5)) → We can import only add function.

④ import calculator as cal
print(cal.add(10,5))

43. Explain `_name_` and `_main_`. Write a correct `'if _name_ == '_main_':` example.

44. Design a package tree with `_init_.py`, nested packages, utilities and tests. Show valid imports.

python_package - dir

↳ src

↳ python_app

↳ `--int--.py`

↳ calculator.py

↳ main.py

↳ tests

- dir

→ src directory

→ dir

↳ modules

→ write your tests

License

Pyproject.toml

Readme.md.

PART F — FUNCTIONS & INTEGRATED CHALLENGES

45. Explain Module vs Package vs Library vs Framework with one Python example for each.

- ① **Module** — A single python file that contain code. Ex: calculator.py
- ② **package** — A directory contains multiple modules (files) Ex: opd - package
- ③ **Library** — A collection of re-usable code / module / package. Ex: python api
- ④ **Framework** — A structure that fix flow of application. Django, playwright.

46. Write a function using default arguments, positional arguments, keyword arguments, *args and **kwargs.

```
def Student (name, age=18, *args, **kwargs):
    print ("Name: ", name)
    print ("Age: ", age)
    print ("Extra: ", args)
    print ("Details: ", kwargs)
```

① positional argument ② Default Argument

Student ("Pavan", 34) Student ("Pavan")

③ keyword argument Student (name="Pavan", age=34)

④ *args Student ("Pavan", 34, "Maths", "Science")

⑤ **kwargs

47. Write a recursive function to calculate factorial and explain the base case.

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)
```

Print (factorial(5)) o/p: 120

48. Write a program using if/elif/else, nested if, for, while, range(), break, continue and pass in a meaningful mini problem.

```
marks = [85, 72, -5, 45, 110, 90]
```

① # For loop + range()

```
for i in range(len(marks)):
    mark = marks[i]
```

② # If else, elif, else

```
if mark < 0 or mark > 100:
    print ("Invalid mark: ", mark)
    continue
elif mark >= 80:
    grade = "A"
elif mark >= 50:
    grade = "B"
else:
    grade = "C"
```

③ # Nested if

```
if mark >= 50:
    if mark >= 80:
        print ("Excellent - A")
    else:
        print (mark, "pass")
```

④ # PASS

```
if True:
    pass
```

⑤ # while loop break

```
count = 1
while count <= 5:
    if count == 3:
        print ("Reached Max limit")
        break
    else:
        count = count + 1
```

49. Build a reusable automation-style utility module that validates input, catches exceptions and returns a clean result.

```
def divide():
```

```
    try:
```

```
        a = int(input("Enter first number:"))
```

```
        b = int(input("Enter second number:"))
```

```
        result = a/b
```

```
        print(result)
```

```
    except ValueError:
```

```
        print("Please enter numbers only")
    except ZeroDivisionError: print("Please cannot divide by zero")
```

50. CAPSTONE: Build a Student Result Analyzer. Input marks, validate bad input, calculate total/average/grade, use functions, a custom exception, a reusable module and a package-style structure. Add a clean report.

① main.py

```
def validate_marks(marks):
```

```
    for mark in marks:
```

```
        if mark > 100:
```

```
            raise ValueError("Enter valid mark")
```

```
def generate_report(subjects, marks, total, average, grade):
```

```
    print("Student Report")
```

① exceptions.py

```
class InvalidMarkError(Exception):
```

```
    pass
```

② calculator.py

```
def calculate_total(marks):
```

```
    return sum(marks)
```

```
def calculate_average(marks)
```

```
    return sum(marks)/len(marks)
```

```
def calculate_grade(average):
```

```
    if average >= 80:
```

```
        return "A"
```

```
    elif average >= 70:
```

```
        return "B"
```

```
    elif average >= 60:
```

```
        return "C"
```

```
    elif average >= 50: return "D"
```

```
    else: return "F"
```

③ generator.py

```
def get_subject_marks(subjects):
```

```
    for subject in subjects:
```

```
        while True:
```

```
            try:
```

```
                mark = float(input)
```

```
                if "Enter mark for
```

```
                (Subject)?"
```

```
            if mark < 0 or mark > 100:
```

```
                raise ValueError("Mark should be between 0 and 100")
```

```
            yield mark
```

```
    except ValueError as e:
```

```
        print("Invalid input: ", e)
```

☐ COMPLETE SYLLABUS COVERAGE

This page is an instructor verification checklist. Every requested topic is represented in the assessment.

- ☑ DATA TYPES: int • float • complex(real/imag/conjugate) • bool • str • list • tuple • set • dict
- ☑ STRING METHODS: upper • lower • capitalize • title • strip • replace • split • join • find • index • count • startswith • endswith • isalpha • isdigit • isalnum • isspace
- ☑ LIST METHODS: append • extend • insert • remove • pop • clear • index • count • sort • reverse • copy
- ☑ SET METHODS: add • remove • discard • pop • clear • union • intersection • difference • symmetric_difference • issubset • issuperset • isdisjoint
- ☑ DICT METHODS: get • keys • values • items • update • pop • popitem • clear • copy • setdefault
- ☑ OPERATORS: arithmetic • comparison • assignment • logical • identity • membership • bitwise
- ☑ CONTROL FLOW: if • if-else • if-elif-else • nested if • multiple conditions • conditional expression • for • while • nested loops • range • collection loops • break • continue • pass • return • yield
- ☑ FUNCTIONS: syntax • parameters • arguments • positional • keyword • default • return • *args • **kwargs • built-in • user-defined • lambda • recursive • nested • higher-order
- ☑ ERROR HANDLING: error vs exception • syntax/runtime/logical errors • try • except • else • finally • multiple exceptions • raise • custom exceptions • hierarchy
- ☑ ITERATORS/GENERATORS: iterable • iterator • iter • next • StopIteration • custom iterator • generator • yield • generator function • expression • generator vs list
- ☑ MODULES/PACKAGES: module • creating modules • import • from import • aliases • __name__ • __main__ • reusable test utilities • package structure • __init__.py • nested packages • imports • automation framework structure
- ☑ LIBRARIES/Frameworks: module vs package • package vs library • library vs framework • standard library

☐ INSTRUCTOR MASTER RUBRIC

Area	Excellent	Good	Needs Practice
Data types & methods	☐	☐	☐
Operators & control flow	☐	☐	☐
Functions	☐	☐	☐
Error handling	☐	☐	☐
Iterators & generators	☐	☐	☐
Modules & packages	☐	☐	☐
Libraries & frameworks	☐	☐	☐
Coding quality	☐	☐	☐

☐ FINAL FEEDBACK / ACTION PLAN

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There are approximately 20 lines visible. On the left side, there is a small metal binder clip holding the paper. The paper appears slightly aged or off-white.